Contents

1 Patterns as Portable Field Programs for Interface Behavior **1**
1.1 Abstract . . . 2
1.2 1. Introduction . . . 2
1.2.1 1.1 The problem: relational behavior is authored as one-off imperative code . . . 2
1.2.2 1.2 The pattern answer . . . 3
1.2.3 1.3 Contributions . . . 3
1.3 2. Background and related work . . . 3
1.4 3. The pattern schema . . . 4
1.4.1 3.1 The fields . . . 4
1.4.2 3.2 Lane separation, made literal . . . 5
1.4.3 3.3 A real pattern, annotated . . . 5
1.5 4. The conformance gate . . . 6
1.5.1 4.1 Primitives must be real passported tokens . . . 6
1.5.2 4.2 Declared primitives must not drift from the body tokens . . . 6
1.5.3 4.3 A primitive is a token, never another lane . . . 7
1.5.4 4.4 Render and diagnostic layers must be real modes . . . 7
1.5.5 4.5 The accessibility fallback is required . . . 8
1.5.6 4.6 Why this is the same discipline as passports . . . 8
1.6 5. The catalog as a data store . . . 8
1.6.1 5.1 Sixty-four patterns, held as data . . . 8
1.6.2 5.2 Every record is a gate-checked fixture . . . 9
1.6.3 5.3 The count cannot drift: `check:readme` . . . 9
1.6.4 5.4 The point: expressiveness lives in composition, not the engine . . . 9
1.7 6. From pattern to running behavior, and back to explanation . . . 9
1.8 7. Evaluation: an authoring argument . . . 10
1.8.1 7.1 What the model buys . . . 10
1.8.2 7.2 A lightweight authoring-time evaluation plan (a plan, no results) . . . 11
1.8.3 7.3 Honest framing of the breadth . . . 11
1.9 8. Limitations . . . 11
1.10 9. Discussion . . . 12
1.11 10. Conclusion . . . 12
1.12 Appendix A. Reproducibility . . . 12
1.13 Appendix B. Conversion notes (markdown → preprint) . . . 13
1.14 Citation coverage . . . 13

1 Patterns as Portable Field Programs for Interface Behavior

Status: research draft (preprint, work in progress). Paper 6 of the Fundamental family — the authoring-model paper. Claims verified against the codebase and canonical docs as of 2026-06-07. See the series index and *the caveat canon* therein. This is a preprint draft, not canonical product documentation.

Terminology note. This paper predates the "Field Pattern" terminology decision. A **Field Formation** is the canonical concept (an authored field arrangement); `FieldPattern` — the subject of this paper — is its current API representation. Read "pattern" throughout as that API representation; the schema, compiler, validator, and catalog names are unchanged. See `../canonical/dimensional-coupling.md`.

Author: Zach Shallbetter

Series: Fundamental Research Papers, Paper 6 of 8 **Companion paper (flagship):** Fundamental: A Field Translation Runtime for Relational DOM Interfaces. This paper goes deep where the flagship's §6.6 (taxonomy hygiene) and §7.3 (pattern runtime) were an overview; it cross-references rather than repeats.

1.1 Abstract

Expressive relational interface behavior — a search result that sinks into a deeper well as its relevance rises, a claim that is weakened by a contradicting source, a section that accumulates a memory of having been read — is normally authored in imperative, one-off code. Such code drifts from its own description, resists inspection, and lets *concept* words leak into the *executable* vocabulary: a developer writes `drag` or `spring` or `orbit` as if the engine had such a force when it does not. We present **Fundamental patterns**: a portable, serializable, **conformance-gated** schema that makes relational behavior *authorable* by separating it into strict, non-overlapping lanes — concepts describe, runtime tokens execute, metrics measure, diagnostics explain, conditions activate, and a *required* reduced-motion equivalent guarantees the behavior is never motion-only. The contribution is threefold: (1) the `FieldPattern` schema, in which these lanes are distinct typed fields; (2) a conformance gate (`validateRecipe`) that mechanically rejects any pattern whose executable tokens are not real passported engine forces, whose render or diagnostic layers are not real modes, whose declared primitives drift from its body tokens, or that lacks its accessibility fallback — so expressive prose can never corrupt the runtime vocabulary; and (3) a shipped catalog of **64** patterns across four sixteen-recipe tiers, held as *data*, that composes a small passported primitive set into a broad, navigable, executable library *with no new engine code*. The 64-recipe breadth is an *existence argument* for the model's expressiveness, not a user study; we sketch an authoring-time evaluation *plan* and are explicit about what the gate does and does not check. This is the authoring model only; we defer the paradigm to Paper 1, runtime execution to Paper 5, accessibility-conformance depth to Paper 4, data binding to Paper 7, and diagnostics to Paper 8.

1.2 1. Introduction

1.2.1 1.1 The problem: relational behavior is authored as one-off imperative code

The flagship paper (Paper 1) reframes the interface as *one shared, inspectable field of meaning* in which DOM elements, particles, relationships, and data records participate as typed agents, and in which *elements bend the field and the field bends them back*. That reframe makes a new class of behavior possible — conserved attention, material typography, cross-boundary causality, memory fields — but it raises an authoring problem that the paradigm paper only sketches: *how does a designer or developer compose such behavior without writing bespoke simulation code each time?*

The default answer in conventional UI engineering is imperative and local. A behavior such as "results settle by relevance and recency" becomes a hand-written tangle of event listeners, easing functions, class toggles, and timers, owned by one component. Three pathologies follow.

1. **Drift.** The code and its description diverge. A comment says "spring," the implementation is a lerp; a name says "physics," the body is a tween. Nothing forces the prose and the behavior to agree, so over time they do not.
2. **Opacity.** The behavior is not *inspectable*. There is no artifact a reviewer can read to learn *which* relations are active, *what* is measured, *how* it degrades under reduced motion, or *which* primitive caused a given effect. The behavior lives only in its execution.
3. **Vocabulary leak.** Most corrosively, *concept* words leak into the *executable* vocabulary. A developer reaches for `drag`, `spring`, `orbit`, `reflect`, `entropy`, or `coherence` and wires them up as if they were forces the engine implements. They are not: in Fundamental, `drag` is the concept that the `viscosity` force realizes, `spring` is realized by `tether`, `reflect` by `wall`, and `entropy`/`coherence` are *metrics*, not forces. When the concept word becomes a callable token, the runtime vocabulary rots — the engine's real, auditable capabilities are buried under invented ones, and the system can no longer make honest claims about what it does.

This third pathology is the one the paradigm cannot tolerate. Paper 1's entire epistemic posture — truth modes, force passports, conformance, lint — exists to keep the boundary between physics, design, and

expression *checkable rather than rhetorical* (Paper 1 §9.2). A free-form authoring layer that lets any word become a force would undo that posture at the seam where authors actually work.

1.2.2 1.2 The pattern answer

A **pattern** is Fundamental's reusable authoring unit: *a portable, serializable, inspectable field program* (Paper 1 §7.3). Rather than imperative code, a pattern is a declarative record with separated lanes — a human intent, an optional natural-field classification, the strict set of *runtime tokens* it composes, its bodies and relationships, its render layers, its metrics, its diagnostics, its conditions, its product-language concepts, and a *required* accessibility fallback. It is data, not behavior; *applying* it builds a scene (Paper 5), but the pattern itself mutates nothing.

The decisive property is that a pattern is **conformance-gated**. A single validator, `validateRecipe`, rejects any pattern whose executable tokens are not real passported engine forces, whose render or diagnostic layers are not real modes, whose declared primitives drift from the tokens its bodies actually carry, or that omits its reduced-motion equivalent. The gate is the mechanism behind the slogan in this paper's core claim: *behavior composes from a small primitive set without corrupting the runtime vocabulary.* Pattern prose may be as expressive as the author likes — "completion releases pressure and decays into memory" — while the pattern's executable `primitives` stay strict (`[morph, memory, gravity, pressure, cohesion]`), because the lanes are typed apart and only one lane is executable.

1.2.3 1.3 Contributions

This paper contributes the **authoring model**:

1. **The `FieldPattern` schema** (§3): a serializable record whose lanes — concepts, runtime tokens, metrics, diagnostics, conditions, render layers, and a required accessibility fallback — are distinct typed fields, so a word's role is declared by *where it lives*, never guessed.
2. **The conformance gate** (§4): `validateRecipe`, which mechanically keeps the runtime vocabulary uncorrupted by rejecting non-token primitives, non-mode render/diagnostic layers, primitive drift, and missing accessibility fallbacks — the same epistemic discipline as force passports (Paper 1 §6.2, Paper 5 §5), applied to *composition*.
3. **The 64-recipe catalog as a data store** (§5): all 64 patterns held as data across four sixteen-recipe tiers, every one gate-checked, demonstrating that a small passported primitive, metric, and render vocabulary composes into a broad, navigable, executable library *with no new engine code* — the expressiveness lives in composition, not in the engine.

The paper is deliberately narrow. The paradigm is Paper 1's; the *execution* of a compiled pattern (the DOM-applying `applyRecipe`, the host, the conformance runtime) is Paper 5's; reading is Paper 2's, evidence is Paper 3's, accessibility-conformance depth is Paper 4's, data binding is Paper 7's, and diagnostics are Paper 8's. We cross-reference and do not re-explain.

1.3 2. Background and related work

Design tokens and design systems. Design-token systems factor visual decisions — color, spacing, type scale — into named, portable values shared across platforms, and design systems compose those tokens into reusable components. Fundamental patterns share the *naming-and-portability* instinct but operate one level up the abstraction: a pattern names a *behavior*, not a value, and the thing being kept portable and consistent is a relational interaction rather than a visual constant. The conformance gate is the pattern analog of a token system's validation that a reference resolves to a real token. [w3c-design-tokens]

Declarative versus imperative UI behavior. A long line of work argues for describing *what* an interface should do rather than *how*, from constraint-based and reactive UI to declarative animation specifications. Patterns are declarative behavior specifications whose distinguishing feature is not the declarativeness itself

but the *typed lane separation* and the *conformance gate* that together prevent the description from drifting from the engine's real capabilities. [react-docs-declarative; fowler2004presentation]

Visual and blocks-based authoring. Block and node-graph authoring environments let non-programmers compose behavior by wiring typed nodes, and refuse connections that violate the type system. A pattern's gate plays the role of that refusal — a primitive that is "really a metric" is rejected the way an incompatible socket is — but patterns are textual, serializable records first; a visual composer is future work (§8, R15). [resnick2009scratch]

Macro and preset systems. Presets and macros expand a compact authored unit into a larger configuration. Fundamental's cosmological *presets* (Paper 1 §6.4 — `blackhole` expands into `attract + swirl + sink + lens`) are exactly such an expansion over the force catalog, and patterns are the broader, schema-backed generalization: a pattern is a *named, validated, multi-lane composite*. The key difference from a macro is the gate: expansion alone does not guarantee the result references only real capabilities; validation does. [fowler2010dsl]

Domain-specific languages for interaction. DSLs for animation, interaction, and state machines give interaction behavior a small, checkable surface. A pattern is best read as a *data-DSL* for relational field behavior, with `validateRecipe` as its type-checker and the passport set (§4) as its symbol table. [fowler2010dsl; myers1990garnet]

The distinguishing stance, across all of these, is the one inherited from the flagship: the authoring layer is *epistemically disciplined*. Every pattern is auditable against the engine's actual capabilities, by running a validator, not by trusting prose.

1.4 3. The pattern schema

The schema is defined in `packages/core/src/recipes/schema.ts` as the `FieldPattern` interface. Its fields are deliberately partitioned into the lanes from Paper 1 §6.6; the schema's own comment names them — *"the lanes — kept separate; a word lives in exactly one."*

1.4.1 3.1 The fields

A `FieldPattern` carries:

- **`id`** — a stable kebab-case identity (`priority-well`), the pattern's handle across docs, gallery, and lookup.
- **`name`**, **`intent`** — the human-facing label and a one-line statement of what the behavior is *for* ("make important elements feel naturally weighted without shouting").
- **`naturalField`** (optional) — the one fundamental field this pattern principally *translates* (`gravity` / `electromagnetic` / `strong` / `weak`), connecting the pattern to the Natural Field Translation System (Paper 1 §6.5). A `translation` phrase carries the conceptual gloss.
- **`primitives`** — *the runtime-token lane.* The strict, real, **passported** engine forces the pattern composes. This is the *only* executable lane, and it must equal exactly the distinct tokens the pattern's bodies carry (§4).
- **`concepts`** (optional) — the product-language lane: human-facing words (`orbit`, `spring`, `trust`, `staleness`) that *describe* the behavior and are **never** runtime tokens.
- **`metrics`** — the measured/semantic-state lane (`density`, `attention`, `confidence`, `coherence`, `entropy`): what the field *measures*, never a force.
- **`diagnostics`** — the inspection-mode lane (`topology`, `causality`, `potential`): the overlays that *reveal* the behavior, never a force.
- **`conditions`** (optional) — the activation lane (`dwell`, `stale`, `in-view`, `focused`): gates that read state, never a force.
- **`bodies`** — the authored elements, each a `BodyRecipe` of one or more space-separated force tokens plus attributes (`strength`, `range`, `spin`, `angle`, `feedback`, `scope`).

- **relationships** (optional) — typed connections between conceptual endpoints (`{ from: 'claim', to: 'source', type: 'supports' }`).
- **render** — the render-layer stack (`particles`, `trails`, `links`, `heatmap`, …).

  Naming note — `particles` (pattern layer) vs `render="dots"` (element attribute). The pattern `RenderLayer` token for the base matter layer is `particles` (valid in `schema.ts`), but the `<field-root>` custom element's render attribute spells the same surface `render="dots"`. They coexist deliberately: the `RenderLayer` lane name predates the element-attribute rename, and the two live in different lanes (pattern data vs. element authoring). A consumer must not write `render="particles"` on `<field-root>` — the element attribute value is `dots`.

- **accessibility** — *required.* An `AccessibilityRecipe` of two non-empty strings: `reducedMotion` (what replaces motion under `prefers-reduced-motion`) and `meaningWithoutMotion` (how meaning survives without color or motion). No pattern is motion-only.
- **budget** (optional, `Partial<PerformanceBudget>`) and **expected** (optional `ExpectedMetrics` — `particleCount`, `entropyRange`, `energyDriftMax`): the performance envelope and the conformance fingerprint a running instance should match.
- **tier**, **status** — the catalog placement (`core` / `applied` / `systems` / `operational`) and implementation status (`shipped` / `experimental` / `planned` / `conceptual`), injected during catalog assembly (§5).
- **notes** — free expressive prose.

1.4.2 3.2 Lane separation, made literal

Paper 1 §6.6 states the discipline as a principle: *concepts describe, runtime tokens execute, metrics measure, diagnostics explain, conditions activate.* The schema makes it *literal* — each lane is a distinct typed field, so a word's role is determined by which field it appears in, and the question "is `orbit` a force?" is answered structurally (`orbit` lives in `concepts`, so no) rather than by inspection of an implementation. This is the schema-level realization of the slogan *natural fields are not tokens; tokens are translations* (Paper 1 §6.5): the pattern can *say* "orbit" all it likes in `concepts` and `notes`, while the executable `primitives` stay `[attract, magnetism, tether]`.

1.4.3 3.3 A real pattern, annotated

The `Evidence Field` pattern (`catalog.ts`, the `EVIDENCE_FIELD` record — also a first-release pattern and the substrate for Paper 3) reads, in source, as follows — the assembled record additionally gains a `tier`, a `status`, and a `conditions` lane at catalog assembly (§5.1):

```
export const EVIDENCE_FIELD: FieldPattern = {
  id: 'evidence-field',
  name: 'Evidence Field',
  intent: 'show how sources support, weaken, or contradict a claim',
  naturalField: 'electromagnetic',
  primitives: ['charge', 'link', 'cohesion', 'repel'],
  bodies: [
    { body: 'charge', strength: 0.9, range: 280, spin: 1 },
    { body: 'link', strength: 0.7, range: 320 },
    { body: 'cohesion', strength: 0.6, range: 260, feedback: true },
    { body: 'repel', strength: 0.5, range: 200 },
  ],
  relationships: [{ from: 'claim', to: 'source', type: 'supports', strength: 0.7 }],
  render: ['links', 'particles', 'heatmap'],
  metrics: ['coherence', 'entropy'],
  diagnostics: ['topology', 'causality', 'links'],
  accessibility: {
    reducedMotion: 'a static claim/source table with support and conflict badges',
```

```
    meaningWithoutMotion: 'each source is listed as supporting or contradicting, with a confidence label
  },
  notes: 'Claims are bodies; supporting sources bind them (link + cohesion), contradictory sources repel
};
```

Reading it by lane:

- **Concept / intent / notes (describe).** The intent and notes are expressive: sources "support, weaken, or contradict"; "strong evidence increases coherence." These words orient a human and never touch the engine.
- **Runtime tokens (execute).** `primitives: ['charge', 'link', 'cohesion', 'repel']` — exactly the distinct tokens the four `bodies` carry, every one a real passported force. *This* is what runs.
- **Metrics (measure).** `coherence` and `entropy` are *measured* state, not forces. They become feedback-variable bindings at compile time (`--field-coherence`, `--field-entropy`; §6), never `data-body` tokens. The notes can claim entropy "rises," but entropy is never *applied*.
- **Diagnostics (explain).** `topology`, `causality`, `links` are inspection overlays that reveal the behavior; they are not forces either.
- **Relationships.** The `claim → source supports` edge is the pattern's relational structure, resolved against real bodies only when the pattern is applied (Paper 5, Paper 7).
- **Accessibility (required).** A static claim/source table with support and conflict badges is the reduced-motion equivalent; the confidence label is how meaning survives without motion. The schema *requires* both strings to be present and non-empty (§4).

The same reading applies to `GUIDED_FLOW` (`primitives: ['magnetism', 'fieldflow', 'stream', 'propagate']`), whose notes say "magnetism bends, fieldflow carries" — expressive prose that names the real, conformance-tested distinction (Paper 1 §3.3) without inventing a "carry" force.

1.5 4. The conformance gate

The central contribution is the gate. `validateRecipe(r: FieldPattern): RecipeProblem[]` (`packages/core/src/recipes/schema.ts`) returns *every* structural problem with a pattern; an empty array means valid. It is what makes "without corrupting the runtime vocabulary" *mechanical* rather than aspirational. We walk its checks in order, quoting the implementation.

1.5.1 4.1 Primitives must be real passported tokens

Every token a body carries is looked up in the passport registry; an unknown token is rejected:

```
(r.bodies ?? []).forEach((b, i) => {
  const tokens = (b.body ?? '').split(/\s+/).filter(Boolean) as Token[];
  if (tokens.length === 0) problems.push({ path: `bodies[${i}].body`, issue: 'empty force token list' }
  for (const t of tokens)
    if (!passportFor(t)) problems.push({ path: `bodies[${i}].body`, issue: `unknown force token "${t}"`
});
```

`passportFor` (`packages/core/src/contracts/passport.ts`) resolves a token only if it has a *passport* — the machine-readable, conformance-cross-checked declaration of what the force is (Paper 1 §6.2). The passport registry is the pattern gate's symbol table. A token like `wormhole` has no passport, so it can never appear in a body; the test suite asserts exactly this rejection.

1.5.2 4.2 Declared primitives must not drift from the body tokens

The `primitives` lane is not free-form: it must equal *exactly* the distinct tokens the bodies carry, in neither direction more nor less.

```
const derived = primitivesOf(r.bodies ?? []);
const declared = r.primitives ?? [];
if (declared.length !== derived.length || derived.some((t) => !declared.includes(t)) || declared.some((
  problems.push({ path: 'primitives', issue: `must list exactly the body tokens (expected: ${derived.jo
```

This closes the gap between a pattern's *self-description* and what it actually does. A pattern cannot advertise `primitives: ['attract', 'swirl']` while its bodies only carry `attract` (the test suite exercises precisely this drift). The declared executable surface and the real executable surface are forced to coincide.

1.5.3 4.3 A primitive is a token, never another lane

The gate then applies a *cross-lane guard*: a declared primitive must be a real runtime token and can *never* be a diagnostic, metric, condition, or concept. The error is lane-aware:

```
declared.forEach((p, i) => {
  if (passportFor(p as Token)) return;
  const lane = OTHER_LANE[p];
  problems.push({
    path: `primitives[${i}]`,
    issue: lane ? `"${p}" is a ${lane}, not a runtime token - move it to ${lane === 'phase' ? 'a schedul
  });
});
```

`OTHER_LANE` is a curated map of well-known words that belong to a *different* lane: `potential`, `topology`, `causality`, `energy` (diagnostics); `mass`, `attention`, `confidence`, `coherence`, `entropy`, `priority` (metrics); `spring`, `orbit`, `drag`, `reflect`, `decay`, `staleness` (concepts); `dwell`, `stale`, `in-view`, `focused` (conditions); and the scheduler phases. The map's own comment is emphatic that its *keys must never be real tokens* — `pressure`, `memory`, and `gate` are deliberately *absent* from it because they really are forces. The map exists only to turn a generic "unknown token" error into a *lane-aware* one: feed `potential` into `primitives` and the gate says *"'potential' is a diagnostic, not a runtime token — move it to diagnostics."* The test suite verifies this lane-awareness for one word per lane (`potential`→diagnostic, `mass`→metric, `orbit`→concept, `dwell`→condition).

This is the precise mechanism behind the claim. The pattern author may write expressively — and the catalog does: `Focus Orbit`'s concept is literally `orbit`, `Search Relevance Field` speaks of recency, and the `completion-release` pattern (`[morph, memory, gravity, pressure, cohesion]`) describes completion as "a weak-force phase change (`morph`) that releases the tension and leaves a memory mark instead of disappearing." But `orbit` can never become a force (it is a concept), `decay` can never become a force (it is a concept), and `entropy`/`coherence` can never become forces (they are metrics). *The prose is free; the token lane is sealed.*

1.5.4 4.4 Render and diagnostic layers must be real modes

```
(r.render ?? []).forEach((layer, i) => {
  if (!RENDER_LAYERS.has(layer)) problems.push({ path: `render[${i}]`, issue: `unknown render layer "${
});
(r.diagnostics ?? []).forEach((mode, i) => {
  if (!FIELD_MODES.has(mode)) problems.push({ path: `diagnostics[${i}]`, issue: `unknown diagnostic mod
});
```

`RENDER_LAYERS` is the nine-member set of matter/structure/scalar surfaces; `FIELD_MODES` is `RENDER_LAYERS` ∪ `DIAGNOSTIC_MODES` (`particles` is already a member of `RENDER_LAYERS`). A test asserts that `FIELD_MODES` covers *every* mode in the live `RENDER_MODES` visualization catalog plus `particles` (the base layer that catalog omits), so a pattern can never reference an overlay the renderer cannot produce, and the recipe-referenceable mode set cannot silently drift from the renderer's real one.

1.5.5 4.5 The accessibility fallback is required

```
if (!r.accessibility || !r.accessibility.reducedMotion || !r.accessibility.meaningWithoutMotion)
  problems.push({ path: 'accessibility', issue: 'reducedMotion + meaningWithoutMotion are required (no p
```

Both strings must be present and non-empty. A pattern without a reduced-motion equivalent does not pass the gate and therefore cannot enter the catalog. The depth of the reduced-motion *conformance* model — semantic source, visual–semantic binding, static equivalent, the lint rules — is Paper 4's; here the load-bearing fact is only that the equivalent is *required at authoring time*. (`naturalField`, `tier`, and `status`, when present, are likewise checked against their closed sets.)

1.5.6 4.6 Why this is the same discipline as passports

The gate is the recipe-level analog of the force passport (Paper 1 §6.2, Paper 5 §5). A passport keeps a *force* honest by cross-checking its declared physics against the live registry and the conformance catalog, so a passport cannot drift from the force it describes. The gate keeps a *pattern* honest by cross-checking its declared composition against the passport registry and the render-mode catalog, so a pattern cannot reference a capability the engine lacks. In both cases the honesty is *mechanical* — a test runs and fails — which is the posture the whole family adopts toward its own claims (Paper 1 §10, "what the epistemics buy"). The pattern gate extends that posture from individual forces to their *composition*, which is exactly where an authoring layer is most tempted to cheat.

1.6 5. The catalog as a data store

1.6.1 5.1 Sixty-four patterns, held as data

`packages/core/src/recipes/catalog.ts` (~1,550 lines) is *the data store*: it holds all **64** `FieldPattern` records as plain data — not code branches, not behavior — across four tiers of sixteen patterns each:

```
export const RECIPE_TIERS: readonly RecipeTierGroup[] = [
  { key: 'core',         label: 'Core interface & accessibility',        patterns: decorate(TIER_CORE,
  { key: 'applied', label: 'Applied - product, workflow & collaboration', patterns: decorate(TIER_PRODU
  { key: 'systems', label: 'Systems - safety, provenance & governance', patterns: decorate(TIER_SYSTEMS
  { key: 'operational', label: 'Operational - multi-actor, adaptive & live', patterns: decorate(TIER_EN
];

export const FIELD_RECIPES: readonly FieldPattern[] = RECIPE_TIERS.flatMap((t) => t.recipes);
```

The four named tiers (`TIER_CORE`, `TIER_PRODUCT`, `TIER_SYSTEMS`, `TIER_ENTERPRISE`) each hold exactly sixteen records; `FIELD_RECIPES` is their flattened concatenation in tier order. A `decorate` step injects each record's `tier` and `status` (defaulting to `shipped`) and layers in the per-id `concepts` and `conditions` lanes during assembly. The test suite asserts the invariants directly: `FIELD_RECIPES.length === 64`, ids are unique and kebab-case, each tier holds 16, the tiers concatenate in declared order, and every record carries an injected tier and `shipped` status.

A `recipeById(id)` lookup resolves any pattern by handle, and `FIRST_RELEASE_RECIPE_IDS` stars **8** patterns as the recommended front door — `priority-well`, `signal-path`, `relationship-bond`, `reading-field`, `evidence-field`, `coherence-field`, `memory-trace`, `guided-flow` — chosen to explain the system quickly and to span the four fundamental fields. (A test asserts the set is exactly eight and that all eight resolve.) The catalog spans gravity-family priority patterns (`Priority Well`, `Focus Orbit`, `Search Relevance Field`), electromagnetic signal patterns (`Signal Path`, `Evidence Field`, `Guided Flow`), strong-interaction binding patterns (`Relationship Bond`, `Concept Cluster`, `Coherence Field`), weak-interaction transformation patterns (`Decay Notice`, `Phase Shift`), the `Reading Field` and `Memory Trace`, the platform-layer `Diagnostic Lens`, and the contract pattern `Accessibility Equivalence`, before broadening through applied, systems, and operational patterns.

1.6.2 5.2 Every record is a gate-checked fixture

The catalog's file comment states the property exactly: *"every runtime token is a real passported force, every render layer + diagnostic is a known mode, the declared primitives match the body tokens, and no primitive is a diagnostic/metric/concept/condition — so `validateRecipe` passes for all of them."* The pattern test suite enforces it: it runs `validateRecipe` over every one of the 64 and asserts zero problems, asserts that no primitive is ever a render/diagnostic mode, and asserts that declared primitives equal the distinct body tokens for every recipe. The catalog is therefore not just documentation — it is sixty-four passing conformance fixtures. A record that referenced a non-existent token or omitted its accessibility fallback would fail the build.

The catalog is also frozen. The 64-recipe `FIELD_RECIPES` catalog is a locked set: new patterns are added to a separate `EXPERIMENTAL_RECIPES` collection rather than the gated catalog, so the four-tier, sixteen-each structure cannot drift. This is a *collection*-level freeze and is orthogonal to the per-record `status` lane: a pattern inside the gated catalog may carry `status: 'experimental'` (implementation-maturity metadata, §3.1), which is distinct from membership in the `EXPERIMENTAL_RECIPES` collection (catalog placement). A record can therefore be a frozen-catalog member *and* status-experimental at once; the two lanes answer different questions.

1.6.3 5.3 The count cannot drift: `check:readme`

The "64" is itself guarded. `scripts/check-readme.mjs` (run as `pnpm check:readme`, wired into CI) imports the built core and compares the README's stated catalog counts against the *live* arrays:

```
const COUNTS = [
  ['forces', len(core.MANUAL_FORCES)],
  ['presets', len(core.MANUAL_PRESETS)],
  ['formations', len(core.FORMATIONS)],
  ['render modes', len(core.RENDER_MODES)],
  ['patterns', len(core.FIELD_RECIPES)],
];
```

If the README says "64 patterns" but `FIELD_RECIPES.length` is anything else, the check fails the build with an explicit message. So the catalog's headline number is a *living* fact derived from the code, not a claim that can rot — the same "living docs" discipline the project applies to its force, preset, and render-mode counts.

1.6.4 5.4 The point: expressiveness lives in composition, not the engine

The catalog's own comment is unambiguous that these records *"compose existing primitives and add NO new engine behavior."* This is the substantive claim of the section. Sixty-four product-level behaviors — priority wells, evidence fields, conflict fields, reading fields, attention weather, diagnostic lenses — are assembled entirely from the small passported vocabulary of forces, metrics, and render modes the engine already ships (Paper 1 §6.4). No pattern adds a `field()` or `apply()` implementation; each is a *composition* over the existing catalog, exactly as the cosmological presets compose primitives into cosmology *with no new engine code* (Paper 1 §6.4). The expressive surface is large; the executable surface stays small and audited. That gap — broad library, narrow uncorrupted vocabulary — is the whole argument of the paper, and the catalog is its evidence.

1.7 6. From pattern to running behavior, and back to explanation

A pattern is simultaneously *authorable*, *inspectable*, and *explainable*. We keep the *execution* details deferred to Paper 5 and show only that the same record serves all three roles.

Compilation (authorable → runnable). `compileRecipe(r)` (`packages/core/src/recipes/compile.ts`) is a *pure* function turning a validated pattern into a `CompiledRecipe` runtime plan — "the bridge from

recipe-as-record to recipe-as-program." Crucially, it *preserves the lane split*, as its own comment states: *"concepts describe · tokens execute · metrics measure · diagnostics explain · conditions activate."* Only the `primitives` lane becomes `data-body` behavior; each body compiles to its `data-*` attribute set and its token list. Metrics become feedback-variable bindings — `attention → --field-attention` via `metricVar` — never tokens. The accessibility block compiles into a `RecipeReducedMotionPlan` with concrete `staticOutputs` (metric badges, relationship list, inspector table, a reduced-motion note), so the reduced-motion path is a real output plan, not just prose. The test suite verifies, for all 64, that compiled tokens are all real, that feedback covers the metrics, that a reduced-motion output path is produced, and that *no concept word ever appears as a token.* The DOM-applying counterpart `applyRecipe` lives in `@fundamental-engine/dom` and is Paper 5's subject.

Authoring across surfaces. `recipeToMarkup` and `recipeAuthoring` emit a pattern's copy-paste authoring as native-HTML `[data-body]` markup, a `<field-root>` web-component snippet, and a React `<FieldField>` component — the one compiled contract across three surfaces from Paper 1 §7.2.

The designer door (intent compilation). Below patterns sit *intents*: `compileIntent('draw-focus')` maps a designer-level verb to concrete force tokens (`draw-focus → attract + feedback`, `warn → repel + thermal`). The intent table (`intent.ts`) was corrected in #224 to emit **real tokens only** — its comment is explicit: *"Tokens use the current names (viscosity, wall — not the legacy drag/reflect)."* So `contain-energy` compiles to `viscosity + wall`, not the concept words `drag` and `reflect`. A test asserts every intent preset compiles to passported tokens; an unknown intent returns `null` rather than a silent default. The intent compiler is thus a second, designer-level door that obeys the same token discipline as the gate.

Explanation (record → prose). `explainScene(r)` (`explain.ts`) walks a pattern's bodies, looks up each token's passport, and renders plain language *grounded in the real physics*: it names each layer by its passport `designUse`, and when `fieldflow` is present it appends the transport caveat — *"Matter follows the field geometry because of fieldflow, not the field-owning force itself"* — exactly the distinction Paper 1 §3.3 forbids collapsing. Because the explanation reads from passports, it cannot describe a behavior the pattern does not actually have. The same record is authored, compiled, and explained from one source of truth; explainability depth is Paper 8's.

1.8 7. Evaluation: an authoring argument

This is a design-systems and authoring paper. It makes no user-study claim (caveat canon item 6); the argument is about what the *model* buys, plus a sketched authoring-time evaluation *plan* with no results.

1.8.1 7.1 What the model buys

Authoring without touching engine code. All 64 catalog behaviors are compositions over the existing passported vocabulary; none adds a `field()`/`apply()` implementation (§5.4). A new behavior is authored by writing a record — choosing tokens, metrics, render layers, conditions, and an accessibility fallback — not by extending the engine. The engine's audited surface stays fixed while the library grows.

Auditability by construction. Because every pattern is gate-checked and the catalog is sixty-four passing fixtures (§5.2), the catalog *cannot* reference a non-existent token or mode, *cannot* declare primitives that drift from its bodies, and *cannot* ship a motion-only behavior. These are not review conventions; they are build-failing assertions. A reviewer can trust the catalog's vocabulary the way they can trust that the core reaches no DOM global (Paper 1 §4.2): by the test, not the prose.

Reduced-motion equivalence required at authoring time. The schema *requires* both accessibility strings, the gate rejects a pattern lacking them, and `compileRecipe` produces a concrete reduced-motion output plan for all 64. Accessibility is therefore an authoring-time obligation, not a later retrofit — which connects directly to Paper 4's conformance model (deferred there for depth).

1.8.2 7.2 A lightweight authoring-time evaluation plan (a plan, no results)

Framed strictly as a protocol — *no results are reported* — a future evaluation could probe the model's two promises against the project's "authoring time" target:

- **Time-to-author.** Give developers a target behavior described in prose ("results settle by relevance and recency") and measure wall-clock time to a passing pattern, comparing the pattern path against an imperative baseline. The hypothesis is that the constrained, lane-typed surface lowers time-to-first-correct-behavior.
- **Lane-violation error rate.** Instrument `validateRecipe` during authoring and count how often authors attempt a lane violation (a concept/metric/diagnostic in `primitives`, a primitive drift, a missing accessibility fallback) and how quickly the lane-aware error resolves it. The hypothesis is that the gate *converts* a class of latent runtime bugs (vocabulary leak) into immediate, self-explaining authoring-time errors.
- **Expressiveness coverage.** Sample real interface behaviors from existing products and record what fraction are expressible as a valid pattern without new engine code, and which require a genuinely new primitive — locating the boundary of the compositional surface.

We make no claim about outcomes; these are designs and hypotheses only.

1.8.3 7.3 Honest framing of the breadth

The 64-recipe catalog is an **existence argument**, not a user study. It demonstrates that a small passported primitive/metric/render vocabulary *does in fact* compose into a broad, navigable, executable library that passes the gate — which is a real and checkable property. It does *not* demonstrate that the patterns are easy to author by non-authors of this project, nor that the resulting behaviors help users; those are the open questions §7.2 and Papers 3–4 address. We state the breadth as what it is.

1.9 8. Limitations

The gate checks structure, not feel. `validateRecipe` verifies *structural* validity — real tokens, real modes, matching primitives, a present accessibility fallback. It says nothing about whether a pattern *feels* right, whether its tuned strengths and ranges produce a legible result, or whether the chosen composition actually matches the stated intent. A pattern can be perfectly valid and behaviorally poor. Perceptual quality is out of scope for a structural gate.

Meaning-preservation is not verified. Relatedly, the gate confirms that a `meaningWithoutMotion` string *exists*; it does not and cannot verify that the static equivalent is *perceptually meaning-preserving*. That is precisely the question Paper 4's accessibility study is designed to answer; the gate is necessary, not sufficient.

`meaningWithoutMotion` is currently free prose. The accessibility fallback is two required strings. That guarantees an equivalent was *authored*, but the strings are not yet a structured, machine-checked specification of the static surface (beyond the coarse `staticOutputs compileRecipe` derives from which lanes are present). Full automation of the reduced-motion equivalent — generating and validating the static surface from structure alone — is a gap and future work.

No visual authoring tool yet. Patterns are textual, serializable records. A visual composer that lets non-programmers assemble patterns with the gate enforced live (refusing a metric dropped into the token lane the way a node graph refuses an incompatible socket) is roadmap, not shipped (R15, and Paper 1 §9.1's "visual authoring tools" frontier).

Breadth is not a study. As stated in §7.3, the 64 patterns are an existence argument for compositional expressiveness, not measured evidence of authoring ergonomics or user benefit.

1.10 9. Discussion

Why lane hygiene plus a gate matter for any behavior-authoring system. The deeper lesson generalizes past Fundamental. Any system that lets authors *compose* behavior from a primitive set faces the same temptation: a convenient word — `drag`, `spring`, `orbit`, `entropy` — gets wired up as if it were a primitive, and the executable vocabulary slowly fills with invented capabilities the engine does not actually have. We call this *vocabulary rot* or *token soup*: the symbol table stops corresponding to the implementation, and the system can no longer make honest claims about what it does. Two properties, together, prevent it. First, **strict lanes**: every word has exactly one role, declared by where it lives, so the description can be arbitrarily expressive without any of that expressiveness reaching the executable surface. Second, a **conformance gate**: the executable lane is validated against the engine's *real* symbol table (here, the passport registry and render-mode catalog), so a word that is not a real capability cannot become one. Lane hygiene makes the separation *possible*; the gate makes it *enforced*. Neither alone suffices — lanes without a gate are a convention authors will violate; a gate without lanes has nothing clean to validate.

Fundamental's patterns are one instantiation, but the pattern is portable to any token-and-composition authoring model: design-token systems, animation DSLs, node-graph editors, low-code platforms. The recurring payoff is the one the whole family argues for — a system whose honesty is *mechanical*. A reviewer can confirm that the authoring catalog references only real engine capabilities by running a validator, not by trusting the documentation.

1.11 10. Conclusion

Complex relational interface behavior need not be authored as drifting, opaque, vocabulary-leaking imperative code. Fundamental makes it *authorable* through a portable, serializable, **conformance-gated** pattern schema that separates the behavior into strict lanes: concepts describe, runtime tokens execute, metrics measure, diagnostics explain, conditions activate, and a required reduced-motion equivalent guarantees the behavior is never motion-only. The conformance gate, `validateRecipe`, mechanically keeps the runtime vocabulary uncorrupted — rejecting any pattern whose executable tokens are not real passported forces, whose layers are not real modes, whose primitives drift from its bodies, or that lacks its accessibility fallback — so expressive prose can never invent a force. The shipped catalog of 64 gate-checked patterns, held as data across four sixteen-recipe tiers and assembled entirely from the existing passported vocabulary with no new engine code, is the existence argument that a small primitive set composes into a broad, navigable, executable library. We have been candid that the gate checks structure rather than feel, that meaning-preservation needs Paper 4's study, that the accessibility fallback is still free prose, and that breadth is not a user study. The authoring model defers execution to Paper 5, accessibility-conformance depth to Paper 4, data binding to Paper 7, and diagnostics to Paper 8; this paper's one claim is that behavior can be made portable and authorable *without corrupting the runtime vocabulary*.

1.12 Appendix A. Reproducibility

Every claim in this paper is checkable against the repository.

- **The schema and the gate:** `packages/core/src/recipes/schema.ts` — the `FieldPattern` interface, the `RenderLayer` / `DiagnosticMode` / `FIELD_MODES` sets, the `OTHER_LANE` lane map, `primitivesOf`, and `validateRecipe` (the conformance gate; §4).
- **The catalog as data:** `packages/core/src/recipes/catalog.ts` — `TIER_CORE` / `TIER_PRODUCT` / `TIER_SYSTEMS` / `TIER_ENTERPRISE` (16 each), `RECIPE_TIERS`, `FIELD_RECIPES = RECIPE_TIERS.flatMap(...)` (length 64), `FIRST_RELEASE_RECIPE_IDS` (8), `recipeById`.
- **Compilation and authoring:** `packages/core/src/recipes/compile.ts` — `compileRecipe`, `metricVar`, `recipeToMarkup`, `recipeAuthoring`. Intent: `intent.ts` (`INTENT_PRESETS`, `compileIntent`, real tokens after #224). Explanation: `explain.ts` (`explainScene`).

- **The token symbol table:** `packages/core/src/contracts/passport.ts` — `PASSPORTS`, `passportFor` (what every pattern primitive is validated against), and `validatePassports` (what keeps the passports themselves honest).
- **The conformance fixtures:** `packages/core/src/recipes/recipes.test.ts` — asserts all 64 validate, the four-tier 16-each structure, lane discipline (no primitive is ever a mode), primitive/body match, the required reduced-motion equivalent, and the eight-recipe first-release set.
- **The count gate:** `scripts/check-readme.mjs` (`pnpm check:readme`) — fails the build if the README's stated pattern count diverges from `core.FIELD_RECIPES.length`.

The canonical design documents corroborate the framing: `docs/canonical/authoring-and-recipes.md` (§5 schema, §7 the 64-recipe catalog + tiers, §4 the intent compiler) and `docs/canonical/natural-fields.md` ("Patterns by meaning").

1.13 Appendix B. Conversion notes (markdown → preprint)

Notation is kept LaTeX-compatible; the inline `ts/js` blocks translate to `listings` directly. Figures referenced in prose but not yet drawn — a lane diagram (concepts/tokens/metrics/diagnostics/ conditions/accessibility as parallel typed fields, §3.2), the gate's decision flow (§4), and the catalog tier map (four tiers × sixteen patterns, §5.1) — are produced at conversion time. External citations are resolved and verified against `references.md` before submission; none are fabricated here.

1.14 Citation coverage

- Design tokens and design-systems literature (§2): [w3c-design-tokens].
- Declarative vs. imperative / reactive / constraint-based UI behavior (§2): [react-docs-declarative; fowler2004presentation].
- Visual and blocks-based programming environments (§2): [resnick2009scratch].
- Macro / preset / template-expansion systems (§2): framed as an analogy and grounded in the recipe/runtime material rather than over-claimed as a separate literature result.
- Domain-specific languages for interaction and animation (§2): [fowler2010dsl; myers1990garnet].